

Introduction to Deep Learning

Assignment 0

September 2026

Note: This assignment is not going to be graded. However, you do need to form groups of three people and submit your work on Brightspace.

Your submission should consist of a report in pdf format (3–5 pages) and neatly organized code (Jupyter notebooks) that you’ve used in your experiments so that we can reproduce your results. You can find more information about report writing on the Brightspace → Assignments section. Do not use compression on the files (`.zip/.7z/.rar`) — submit every file individually.

The purpose of this assignment is mainly for you to get accustomed to the requirements of this course and Brightspace submission system. In case your report has some clear issues, we will provide feedback so that you are more prepared for the graded assignments.

Deadline: Look up on Brightspace → Assignments.

Introduction

The objective of this assignment is to develop and evaluate several algorithms for classifying images of handwritten digits and designing your own neural network from scratch for the XOR problem. You will work with a simplified version of the famous MNIST data set: a collection of 2707 digits represented by 16×16 vectors. The data is split into a training set (1707 images) and a test set (1000 images). These data sets are stored in 4 files: `train_in.csv`, `train_out.csv`, `test_in.csv`, `test_out.csv`, where `in` and `out` refer to the input records (images) and the corresponding digits (class labels), respectively. These files are stored in `data.zip`.

You may find more information about the original problem of handwritten digit recognition, more data sets, and an overview of the accuracies of best classifiers (it is about 99.7%) at <http://yann.lecun.com/exdb/mnist/>.

Task 1: Data dimensionality, distance-based classifiers

The purpose of this task is to develop some intuitions about clouds of points in high-dimensional spaces. In particular, you are supposed to use dimensionality reduction techniques to visualize your data, develop a very simple algorithm for classifying hand-written digits, and compare it to another distance-based classifier.

1. For each digit d , ($d = 0, 1, \dots, 9$), let us consider a cloud of points in 256-dimensional space, C_d , which consists of all training images (vectors) that represent d . For each cloud C_d we can calculate its center, c_d , which is just a 256-dimensional vector of means over all coordinates of vectors that belong to C_d .

Once we have these centers, we can easily classify new images: by calculating the distance from the vector that represents this image to each of the 10 centers, the closest center defines the label of the image. Next, calculate the distances between the centers of the 10 clouds, $\text{dist}_{ij} = \text{dist}(c_i, c_j)$, for $i, j = 0, 1, \dots, 9$. Given all these distances, try to say something about the expected accuracy of your classifier. What pairs of digits seem to be most difficult to separate?

2. Experiment with three dimensionality reduction algorithms: PCA, U-MAP, and t-SNE and apply them to the MNIST data to generate a visualization of the different classes, preferably in 2D. You are free to use any library to do this (preferably `scikit-learn` and `umap-learn` packages from PyPI).

Does the visualization agree with your intuitions and the between-class distance matrix dist_{ij} ?

3. Use the mean pixel values of each digit category obtained in part 1 to implement a Nearest mean classifier. Apply your classifier to all points from the training set and calculate the percentage of correctly classified digits. Do the same with the test set, using the centers that were calculated from the training set.
4. A less naive distance-based approach is the KNN (K-Nearest-Neighbor) classifier (you can either implement it yourself or use the one from the `scikit-learn` package). Repeat the same procedure as in part 3 by using this method. Then, for both classifiers, generate a confusion matrix which should provide a deeper insight into classes that are difficult to separate. A confusion matrix is here a 10-by-10 matrix (c_{ij}) , where c_{ij} contains the percentage (or count) of digits i that are classified as j . Which digits are most difficult to classify correctly? Again, for calculating and visualising confusion matrices you may use the `scikit-learn` package. Describe your findings, and compare the performance of your classifiers on the train and test sets.

Task 2: Implement a multi-class perceptron algorithm

Implement (from scratch) a multi-class perceptron training algorithm. Essentially, a multi-class perceptron represents 10 independent linear classifiers that will each try to predict whether a digit belongs to a particular class. The weight update rules can be found in Chapter 6.1 of the *Understanding Deep Learning* textbook (Simon J.D. Prince). Specifically, refer to equations 6.3 and 6.7.

- Train your network on the training set, try calculating the average loss/accuracy values after every epoch and then plot these values. Additionally, try to create a routine that evaluates your performance (loss/accuracy) on your test set as well and plot these values alongside the training set metrics. As your algorithm is non-deterministic (results depend on how you initialize weights), repeat your experiments a few times to get a feeling of the reliability of your accuracy estimates. Try different weight initialisation strategies and learning rates and see how these hyperparameters influence the training performance.
- Try to make your code efficient. In particular, try to limit the number of loops, using matrix multiplication whenever possible. For example, append a column of ones that will represent the bias to your train and test data. The weights of your network can be stored in a matrix W of size 257×10 . Then the output of the network on all inputs is just a dot product of two matrices: T and W , where T denotes the matrix of all input vectors (one per row), augmented with 1's (biases). To find the output node with the strongest activation use the numpy `argmax()` function. An efficient implementation of your algorithm shouldn't take more than a few seconds to converge on the training set (yes, the training set consists of patterns that are linearly separable so the perceptron algorithm will converge).
- How does the accuracy of this single-layer multi-class perceptron compare to the distance-based methods in task 1?

Task 3: Implement the XOR network and the Gradient Descent Algorithm (Optional)

This is probably the last time in your life that you are asked to implement a neural network from scratch — therefore, have fun! Proceed as follows:

1. Implement a class or a function that simulates a neural network with two inputs, two hidden nodes and one output node. The vector `weights` denotes 9 weights (tunable parameters): Each non-input node has three incoming weights: one connected to the bias node that has value 1, and two other connections that lead from the input nodes to a hidden node or from the two hidden nodes to the output node. Assume that all non-input nodes use the sigmoid activation function.

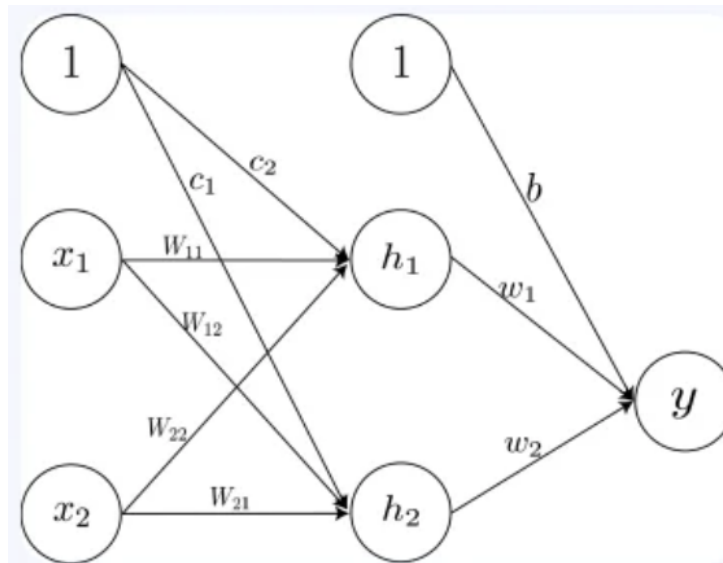


Figure 1: A diagram of a simple neural network with 3 neurons and two layers capable of solving the XOR problem.

2. Implement the error function, which returns the mean squared error made by your network on 4 possible input vectors $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$ and the corresponding targets: 0, 1, 1, 0.
3. Implement a function that calculates the Mean Squared Error (MSE) between the outputs of your network and the category labels of all 4 input vectors. Then implement a function that will calculate the gradients for all the weights in the network. First starting with $[w_1, w_2, b]$ and backpropagating the error to the remaining weights $[w_{11}, w_{12}, w_{21}, w_{22}, c_1, c_2]$.

Please refer to Chapter 7 of *Understanding Deep Learning* for an in-depth explanation about how to calculate gradients and implement weight updates.

4. Finally, implement the gradient descent algorithm:
 - (a) Initialize weights to some random values,
 - (b) Iterate: $\text{weights} = \text{weights} - \eta \cdot \text{gradient}(\text{weights})$, where η is a small positive constant (called “step size” or “learning rate”).

Use your program to train the network on the XOR data. During training, monitor two values: the MSE obtained by your network on the training set, and the number of misclassified inputs. (The network returns a value between 0 and 1; we may agree that values bigger than 0.5

are interpreted as “1”, otherwise as “0”.) Run your program several times using various initialization strategies and values of the learning rate.

Additionally, try the “lazy approach”: just keep generating random weights of the network, testing if it computes the XOR function, and stop as soon as you have found such weights. To get an idea of how many sets of weights should be tried before finding a good one repeat your experiment several times. Describe your work and findings in the report.

You may experiment with alternative activation functions, e.g., hyperbolic tangent ($\tanh$) or a rectified linear unit, $\text{relu}(x) = \max(0, x)$. How do they affect the training process of your network, and how would you explain these differences?